```go
package handlers

import (
"fmt"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/services"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/response"
"github.com/gin-gonic/gin"
"github.com/go-playground/validator/v10"
)

type AgentHandler struct {
agentService *services.AgentService
validator *validator.Validate
}

func NewAgentHandler(agentService *services.AgentService) *AgentHandler {
return &AgentHandler{
agentService: agentService,
validator: validator.New(),
}
}

// CreateAgent godoc
// @Summary Create a new valuation agent
// @Description Create a new valuation agent in the system
// @Tags agents
// @Accept json
// @Produce json
// @Param request body services.CreateAgentRequest true "Create agent request"
// @Success 201 {object} response.Response "Agent created successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /agents [post]
// @Security BearerAuth
func (h *AgentHandler) CreateAgent(c *gin.Context) {
var req services.CreateAgentRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
```

```
response.ValidationError(c, err.Error())
return
}

agent, err := h.agentService.CreateAgent(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Agent created successfully", agent)
}

// GetAllAgents godoc
// @Summary Get all valuation agents
// @Description Get paginated list of all valuation agents
// @Tags agents
// @Produce json
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agents retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /agents [get]
func (h *AgentHandler) GetAllAgents(c *gin.Context) {
// Parse query parameters for pagination
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

agents, total, err := h.agentService.GetAllAgents(limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve agents")
return
```

```
}

data := map[string]interface{}{
"agents": agents,
"total": total,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agents retrieved successfully", data)
}

// GetAgentByID godoc
// @Summary Get valuation agent by ID
// @Description Get specific valuation agent by ID
// @Tags agents
// @Produce json
// @Param id path string true "Agent ID"
// @Success 200 {object} response.Response "Agent retrieved successfully"
// @Failure 404 {object} response.Response "Agent not found"
// @Router /agents/{id} [get]
func (h *AgentHandler) GetAgentByID(c *gin.Context) {
agentID := c.Param("id")

agent, err := h.agentService.GetAgentByID(agentID)
if err != nil {
response.NotFound(c, "Agent not found")
return
}

response.Success(c, "Agent retrieved successfully", agent)
}

// UpdateAgent godoc
// @Summary Update valuation agent
// @Description Update an existing valuation agent
// @Tags agents
// @Accept json
// @Produce json
// @Param id path string true "Agent ID"
// @Param request body services.UpdateAgentRequest true "Update agent request"
// @Success 200 {object} response.Response "Agent updated successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
```

```go
// @Router /agents/{id} [put]
// @Security BearerAuth
func (h *AgentHandler) UpdateAgent(c *gin.Context) {
agentID := c.Param("id")

var req services.UpdateAgentRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

agent, err := h.agentService.UpdateAgent(agentID, req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Agent updated successfully", agent)
}

// DeleteAgent godoc
// @Summary Delete valuation agent
// @Description Soft delete a valuation agent
// @Tags agents
// @Produce json
// @Param id path string true "Agent ID"
// @Success 200 {object} response.Response "Agent deleted successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /agents/{id} [delete]
// @Security BearerAuth
func (h *AgentHandler) DeleteAgent(c *gin.Context) {
agentID := c.Param("id")

err := h.agentService.DeleteAgent(agentID)
if err != nil {
response.BadRequest(c, err.Error())
return
}
```

```
response.Success(c, "Agent deleted successfully", nil)
}

// SearchAgents godoc
// @Summary Search valuation agents
// @Description Search valuation agents by query string
// @Tags agents
// @Produce json
// @Param q query string true "Search query"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agent search completed successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /agents/search [get]
func (h *AgentHandler) SearchAgents(c *gin.Context) {
query := c.Query("q")
if query == "" {
response.BadRequest(c, "Search query is required")
return
}

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

agents, err := h.agentService.SearchAgents(query, limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to search agents")
return
}
```

```
data := map[string]interface{}{
"agents": agents,
"query": query,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agent search completed successfully", data)
}

// GetAgentsByType godoc
// @Summary Get agents by type
// @Description Get valuation agents filtered by agent type
// @Tags agents
// @Produce json
// @Param type path string true "Agent type"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agents by type retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /agents/type/{type} [get]
func (h *AgentHandler) GetAgentsByType(c *gin.Context) {
agentType := c.Param("type")

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

agents, err := h.agentService.GetAgentsByType(agentType, limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve agents by type")
```

```
return
}

data := map[string]interface{}{
"agents": agents,
"agent_type": agentType,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agents by type retrieved successfully", data)
}

// GetAgentsByStatus godoc
// @Summary Get agents by status
// @Description Get valuation agents filtered by status
// @Tags agents
// @Produce json
// @Param status path string true "Agent status"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agents by status retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /agents/status/{status} [get]
func (h *AgentHandler) GetAgentsByStatus(c *gin.Context) {
status := c.Param("status")

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

agents, err := h.agentService.GetAgentsByStatus(status, limit, offset)
```

```
if err != nil {
response.InternalServerError(c, "Failed to retrieve agents by status")
return
}

data := map[string]interface{}{
"agents": agents,
"status": status,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agents by status retrieved successfully", data)
}

// GetAgentsByPartner godoc
// @Summary Get agents by partner
// @Description Get valuation agents filtered by partner ID
// @Tags agents
// @Produce json
// @Param partner_id path string true "Partner ID"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agents by partner retrieved
successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /agents/partner/{partner_id} [get]
func (h *AgentHandler) GetAgentsByPartner(c *gin.Context) {
partnerID := c.Param("partner_id")

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
```

```
}

agents, err := h.agentService.GetAgentsByPartner(partnerID, limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve agents by partner")
return
}

data := map[string]interface{}{
"agents": agents,
"partner_id": partnerID,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agents by partner retrieved successfully", data)
}

// GetAgentWithValuationRequests godoc
// @Summary Get agent with valuation requests
// @Description Get valuation agent with their associated valuation requests
// @Tags agents
// @Produce json
// @Param id path string true "Agent ID"
// @Success 200 {object} response.Response "Agent with valuation requests retrieved
successfully"
// @Failure 404 {object} response.Response "Agent not found"
// @Router /agents/{id}/valuation-requests [get]
func (h *AgentHandler) GetAgentWithValuationRequests(c *gin.Context) {
agentID := c.Param("id")

agent, err := h.agentService.GetAgentWithValuationRequests(agentID)
if err != nil {
response.NotFound(c, "Agent not found")
return
}

response.Success(c, "Agent with valuation requests retrieved successfully", agent)
}

// Helper function to parse integer from string
func parseInt003(s string) (int, error) {
var result int
_, err := fmt.Sscanf(s, "%d", &result)
```

```
return result, err
}
```